

Task 1 — Simulated Brute-Force Attack Using HTML + JavaScript

Create a file called bruteforce.html.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Brute Force Simulation</title>
```

```
</head>
```

```
<body>
```

```
<h2>Login Form</h2>
```

```
<label>Username:</label>
```

```
<input type="text" id="username">
```

```
<br><br>
```

```
<label>Password:</label>
```

```
<input type="password" id="password">
```

```
<br><br>
```

```
<button onclick="login()">Login</button>
```

```
<button onclick="bruteForce()">Simulate Brute Force</button>
```

```
<p id="result"></p>
```

```
<script>
```

```
const correctUsername = "student";
```

```
const correctPassword = "qwerty";
```

```
function login() {
```

```
  const username = document.getElementById("username").value;
```

```
const password = document.getElementById("password").value;

if (username === correctUsername &&
    password === correctPassword) {

    document.getElementById("result").innerHTML =
        "Login successful!";

} else {
    document.getElementById("result").innerHTML =
        "Invalid username or password.";
}
}

function bruteForce() {

    // Only five dummy passwords for this lab
    const passwordList = [
        "123456",
        "password",
        "admin",
        "welcome",
        "qwerty"
    ];

    let found = false;

    for (let password of passwordList) {

        document.getElementById("result").innerHTML =
            "Trying password: " + password;

        if (password === correctPassword) {

            document.getElementById("result").innerHTML =
```

```
        "Password found: " + password;

        found = true;
        break;
    }
}

if (!found) {
    document.getElementById("result").innerHTML =
        "Password was not found.";
}
}

</script>

</body>
</html>
```

Task 2 — Python Dictionary Attack Simulation

Create:

- dictionary_attack.py

Dictionary Attack Simulation

Use only dummy passwords in a local lab.

```
leaked_passwords = [
    "password",
    "123456",
    "qwerty",
    "admin",
    "welcome",
    "letmein",
    "12345678",
```

```
"abc123",  
"password123",  
"admin123"  
]  
  
user_password = input("Enter a password to test: ")  
  
if user_password in leaked_passwords:  
    print("WARNING: Password appears in the simulated leaked-password  
list.")  
    print("Matched password:", user_password)  
else:  
    print("Password was not found in the simulated list.")
```

Run it

- python dictionary_attack.py

Try:

- Enter a password to test: qwerty

Output:

- WARNING: Password appears in the simulated leaked-password list.
Matched password: qwerty

Try something stronger:

- Enter a password to test: MyRandomPassword!9382

Output:

- Password was not found in the simulated list.

Difference from brute force

- Brute force: systematically tries possible combinations.
- Dictionary attack: tries passwords from a predefined list of likely/common/leaked passwords.

Task 3 — Credential Stuffing Simulation

Credential Stuffing Simulation

All credentials below are fictional lab credentials.

```
mock_users = {
    "alice": "Alice@123",
    "bob": "Bob@456",
    "charlie": "Charlie@789"
}

credential_list = [
    ("alice", "wrongpassword"),
    ("bob", "Bob@456"),
    ("admin", "admin123"),
    ("charlie", "wrong123"),
    ("alice", "Alice@123")
]

print("Starting credential-stuffing simulation...\n")

for username, password in credential_list:

    if username in mock_users and mock_users[username] == password:
        print("[SUCCESS]", username, ":", password)
    else:
        print("[FAILED] ", username, ":", password)
```

What is credential stuffing?

- Credential stuffing is different from ordinary brute force.

The attacker uses username/password combinations obtained from another source and attempts to reuse them against another service.

For example:

username	password
alice	wrongpassword
bob	Bob@456
admin	admin123

Task 4 — Account Lockout After 3 Failed Attempts

Create:

- lockout.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Account Lockout Simulation</title>
</head>

<body>

<h2>Login</h2>

<label>Username:</label>
<input type="text" id="username">

<br><br>

<label>Password:</label>
<input type="password" id="password">

<br><br>

<button id="loginButton" onclick="login()">Login</button>
```

```
<p id="message"></p>
```

```
<p id="attempts"></p>
```

```
<script>
```

```
const correctUsername = "student";
```

```
const correctPassword = "Secure@123";
```

```
let failedAttempts = 0;
```

```
const maxAttempts = 3;
```

```
function login() {
```

```
    // Stop login after logout
```

```
    if (failedAttempts >= maxAttempts) {
```

```
        return;
```

```
    }
```

```
    const username =
```

```
        document.getElementById("username").value;
```

```
    const password =
```

```
        document.getElementById("password").value;
```

```
    if (username === correctUsername &&
```

```
        password === correctPassword) {
```

```
        document.getElementById("message").innerHTML =
```

```
            "Login successful!";
```

```
        failedAttempts = 0;
```

```
    } else {
```

```
failedAttempts++;

document.getElementById("message").innerHTML =
    "Invalid username or password.";

document.getElementById("attempts").innerHTML =
    "Failed attempts: " + failedAttempts + " / " + maxAttempts;

if (failedAttempts >= maxAttempts) {

    document.getElementById("message").innerHTML =
        "Account locked after 3 failed attempts.";

    document.getElementById("loginButton").disabled = true;
}
}
}

</script>

</body>
</html>
```

Test it

Correct credentials:

- Username: student
- Password: Secure@123

Incorrect password three times:

- Attempt 1 → Failed
- Attempt 2 → Failed
- Attempt 3 → Account locked

The Login button becomes disabled.